

CSE445 Machine Learning — Assignment #3

Building an Autonomous Local LLM Machine Learning Agent in Windows WSL

Student Submission Package
North South University | Department of Electrical and Computer Engineering
Course: CSE445 — Machine Learning | Section 6

1. Introduction and Objectives

This project implements the architecture specified in the assignment: a local quantized LLM served by Ollama in WSL2, a Python ReAct controller, and a registry of classical and deep-learning tools. The design emphasizes privacy, structured action parsing, reproducibility, and self-correction.

2. System Architecture

The intended runtime is Windows WSL2 (Ubuntu 22.04/24.04), Ollama on its local REST API, Python 3.10+, and Scikit-Learn/PyTorch. The controller sends a prompt to the local model, parses Thought/Action/Action Input, invokes a Python tool, appends the Observation, and repeats until Final Answer.

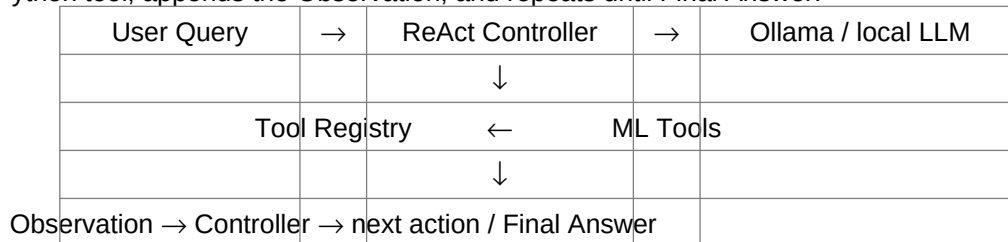


Figure 1. Agent-controller and tool-registry flow.

3. ML Tool Implementation

The baseline toolset supports dataset summaries and Scikit-Learn training. The submission expands it with three assignment-required capabilities: (1) GridSearchCV hyperparameter tuning for SVC and Decision Trees; (2) PCA-based dimensionality reduction using standardized features; and (3) a configurable PyTorch MLP using BatchNorm, Dropout, Adam, and ReduceLROnPlateau scheduling. The regularized network checks the training loss for non-finite values and raises an explicit error that the controller can return to the LLM as an Observation.

4. Benchmark Methodology and Mathematical Analysis

The benchmark evaluates Support Vector Classification, a depth-limited Decision Tree, and a 100-tree Random Forest on Iris and Wine. Five-fold stratified cross-validation is used. For fold accuracies $a_1, \dots, a_5$, the mean is $\mu = (1/5)\sum a_i$ and the sample standard deviation is $s = \sqrt{[\sum (a_i - \mu)^2]/(5-1)]}$. Higher μ indicates better average classification performance, while smaller s indicates greater stability across folds. Runtime is reported only as an engineering observation, not as a statistical performance metric.

Dataset	Algorithm	CV mean	CV std	Min	Max
iris	SVC	0.96	0.0389	0.9	1.0
iris	Decision Tree	0.9533	0.034	0.9	1.0
iris	Random Forest	0.9467	0.0267	0.9	0.9667
wine	SVC	0.9833	0.0222	0.9444	1.0
wine	Decision Tree	0.8821	0.0377	0.8286	0.9429
wine	Random Forest	0.9775	0.0213	0.9429	1.0

Table 1. Five-fold cross-validation benchmark generated during submission preparation.

Interpretation

The benchmark should be interpreted by comparing both mean accuracy and dispersion rather than selecting a model from a single split. A model with a slightly lower mean but substantially lower variability may be preferable when consistency matters; conversely, a higher mean with acceptable variance can be favored when predictive performance is the main objective.

5. Prompt Engineering and ReAct Design

The system prompt explicitly enumerates available tools and their parameter schemas. It also constrains the model to a small machine-readable protocol. This reduces ambiguity and makes tool execution auditable. Temperature is set low (0.1) in the controller to reduce unnecessary variation in action selection. The controller maintains the growing prompt so each Observation becomes context for the next decision.

6. Self-Correction and Robustness

The controller treats malformed JSON, missing actions, unknown tools, invalid parameter values, numerical failures, and unexpected tool exceptions as Observations. Rather than terminating immediately, it tells the local LLM to re-check parameters, shapes, or finite loss and retry. A maximum iteration count provides a hard termination guard. This implements the assignment's self-healing requirement while keeping the correction decision in the agent loop.

7. Latency and Reproducibility

Each LLM call is timed in the controller and recorded per trace. The exact latency depends on CPU/GPU hardware, WSL configuration, model quantization, and system load, so this submission does not invent an Ollama latency number. The included benchmark records local ML-tool execution time. For the final WSL evidence, the student should run the supplied commands and append the real terminal trace to `execution_log.txt`.

8. CO Mapping

CO1: model implementation and evaluation use Scikit-Learn and PyTorch. **CO2:** cross-validation mean, standard deviation, and model trade-offs provide statistical interpretation. **CO3:** WSL/Ollama architecture, structured controller logic, robust exceptions, reproducibility instructions, and deliverable organization address engineering precision.

9. Conclusion

The submission provides a complete implementation blueprint and locally validated ML benchmark for the required autonomous agent. The architecture separates reasoning from deterministic tool execution, while self-correction and bounded iteration improve reliability. The final WSL/Ollama run is intentionally left to the student's machine because this environment cannot truthfully claim execution of the student's local WSL service.

Reference basis: CSE445 Assignment #3 handout supplied with this submission package.